VOL. 31

Cloud (AWS)

A practical map of the AWS services a beginner meets first

Source	aws.amazon.com/training
Generated	2026-10-10
Level	Beginner
Estimated study time	25 hours
Language	English
Volume	Vol. 31 - Cloud (AWS)

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 5

Chapter map 5

Chapter 1 - Cloud fundamentals and the shared responsibility model 6

1.1 Summary 6

1.2 Concepts 7

Three service models 7

Regions, Availability Zones and edge locations 7

The six advantages of cloud computing 8

The shared responsibility model 8

1.3 Examples 9

1.4 Common mistakes and best practices 10

1.5 Practice exercises 10

Solutions 10

1.6 Mini test 11

Answer key and explanations 11

Chapter 2 - Identity and access management: users, roles and policies 12

2.1 Summary 12

2.2 Concepts 12

Principals and identities 12

Users, groups and roles 13

Policy structure 13

Policy types and who they belong to 14

How a request is decided 14

Wildcards and condition keys 15

2.3 Examples 15

2.4 Common mistakes and best practices 16

2.5 Practice exercises 17

Solutions 17

2.6 Mini test 17

Answer key and explanations 18

Chapter 3 - Core compute on EC2 with pricing basics 18

3.1 Summary 18

3.2 Concepts . . . 19
What an instance is . . . 19
Instance types and naming . . . 19
Instance storage . . . 20
The instance lifecycle . . . 20
Metadata and the key pair . . . 20
The five ways to pay . . . 20
3.3 Examples . . . 21
3.4 Common mistakes and best practices . . . 22
3.5 Practice exercises . . . 22
Solutions . . . 22
3.6 Mini test . . . 23
Answer key and explanations . . . 23
Chapter 4 - Storage with S3 and Glacier . . . 23
4.1 Summary . . . 23
4.2 Concepts . . . 24
Objects and buckets . . . 24
Consistency and durability . . . 24
Storage classes . . . 24
Key features . . . 25
Encryption and access control . . . 25
The S3 Glacier archive classes . . . 25
4.3 Examples . . . 26
4.4 Common mistakes and best practices . . . 27
4.5 Practice exercises . . . 27
Solutions . . . 28
4.6 Mini test . . . 28
Answer key and explanations . . . 28
Chapter 5 - Networking with VPC, subnets and security groups . . . 29
5.1 Summary . . . 29
5.2 Concepts . . . 29
CIDR blocks and subnets . . . 29
Routers, gateways and addresses . . . 30
Security groups . . . 30
Network ACLs . . . 30

Security groups versus network ACLs . . . 31
VPC endpoints and flow logs . . . 31
The default VPC . . . 31
5.3 Examples . . . 31
5.4 Common mistakes and best practices . . . 32
5.5 Practice exercises . . . 32
Solutions . . . 33
5.6 Mini test . . . 33
Answer key and explanations . . . 33
Chapter 6 - Databases with RDS and DynamoDB . . . 34
6.1 Summary . . . 34
6.2 Concepts . . . 35
What RDS manages for you . . . 35
High availability and read scaling . . . 35
Backups and snapshots . . . 35
Aurora and ElastiCache . . . 35
DynamoDB basics . . . 35
6.3 Examples . . . 36
6.4 Common mistakes and best practices . . . 37
6.5 Practice exercises . . . 37
Solutions . . . 38
6.6 Mini test . . . 38
Answer key and explanations . . . 39

How to use this volume

Prerequisites. No cloud experience is assumed. You should be comfortable with a command line, know what an IP address and a port are, and be able to read a little JSON. An AWS account with an administrator you trust is useful for the hands-on parts, because everything here can be reproduced in the AWS Management Console while the Free Tier is active.

Estimated study time. About 25 hours: two hours per chapter, plus the time you spend clicking in the console. Chapters 1 to 6 are the core that every AWS role needs. Chapters 7 to 11 cover the operational and cost side of a running system.

How the chapters work. Every chapter opens with a summary, explains the concepts with plain words plus diagrams and tables, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Service names, consoles and the AWS Command Line Interface (AWS CLI) are used throughout. The environment that wrote these notes has no AWS CLI and no network, so every `bash` block is illustrative and every `text` block labelled output is example output that was NOT executed here. On your own machine, install the AWS CLI, configure credentials, and run the same commands against your account.

shell

```bash
# Illustrative only. Nothing in this block was executed while writing.
# Run it on your own machine with the AWS CLI v2 installed and credentials set.
aws configure set region eu-west-1
aws sts get-caller-identity
```

output

```text
(illustrative output, not executed in the writing environment)
{
    "UserId": "AIDAEXAMPLEUSERID00000",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:user/example"
}
```

Account numbers, user names and region names are placeholders; every value that identifies a real resource is shown as `EXAMPLE`.

About the source. The syllabus follows the free, public learning paths that AWS publishes for beginners on `aws.amazon.com/training`, its digital training pages, and its Getting Started tutorials. AWS course text and whitepapers are copyrighted, so every explanation here is written in original words.

Chapter map

#	Chapter	You will learn
1	Cloud fundamentals and the shared responsibility model	what cloud computing is, Regions and Availability Zones, the six advantages, and who secures what
2	Identity and access management: users, roles and policies	IAM users, groups, roles, the JSON that forms a policy, and how a request is decided
3	Core compute on EC2 with pricing basics	AMIs, instance families, the instance lifecycle, storage options, and the five ways to pay
4	Storage with S3 and Glacier	buckets, objects, storage classes, versioning, lifecycle rules and archive retrieval
5	Networking with VPC, subnets and security groups	CIDR blocks, public and private subnets, internet and NAT gateways, security groups and network ACLs
6	Databases with RDS and DynamoDB	managed relational databases, Multi-AZ and read replicas, and the key-value store DynamoDB
7	Load balancing and auto scaling	application and network load balancers, target groups, health checks and auto scaling groups
8	Serverless with Lambda and API Gateway	functions, handlers, limits and triggers, and publishing HTTP APIs through API Gateway
9	Observability with CloudWatch	metrics, logs, alarms and dashboards, and how to watch cost and performance
10	Cost management on AWS	Cost Explorer, budgets, cost allocation tags and the levers that shrink a bill
11	Well-Architected Framework basics	the six pillars, design principles and running a workload review
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Cloud fundamentals and the shared responsibility model

1.1 Summary

- Cloud computing delivers IT resources over the internet on demand, with prices that follow a pay-as-you-go model.
- AWS groups capacity into Regions, and every Region contains at least two, usually three or more, Availability Zones.

- An Availability Zone is one or more data centres with independent power, cooling and networking, linked by low-latency connections.
- Edge locations and Regional edge caches sit close to users to serve content and answer DNS through services such as CloudFront and Route 53.
- AWS promotes six advantages: variable instead of fixed cost, economies of scale, capacity on demand, speed and agility, no data centre upkeep, and global reach in minutes.
- The shared responsibility model divides security into security OF the cloud, held by AWS, and security IN the cloud, held by the customer.
- AWS answers for the physical and host layer: the buildings, the hardware, the network and the virtualisation that isolates instances.
- The customer answers for the guest operating system, the application, the data, identities and permissions, and the encryption settings.
- The more a service is managed by AWS, the smaller the customer share becomes, moving from EC2 to RDS to Lambda.
- The model is not a one-time reading; it is a checklist you return to whenever you add a new service.

1.2 Concepts

Three service models

Every AWS service sits on a spectrum of how much of the stack AWS runs for you.

Model	AWS provides	You manage	Examples
IaaS	virtual machines, storage, networking	guest OS, runtime, application, data	EC2, VPC, EBS
PaaS	a managed platform and runtime	code, data, permissions	Elastic Beanstalk, RDS
SaaS	the whole application	settings and your data	Amazon WorkMail

Regions, Availability Zones and edge locations

A Region is a geographic area, for example `eu-west-1` in Ireland. Each Region is isolated and holds multiple Availability Zones. Deploy across at least two zones when high availability matters, so the loss of one zone does not remove the service.

output

```
AWS Cloud
  |
  |-- Region eu-west-1 (Ireland)
  |     |-- Availability Zone eu-west-1a
  |     |-- Availability Zone eu-west-1b
  |     `-- Availability Zone eu-west-1c
  |
  |-- Region us-east-1 (N. Virginia)
  |     `-- Availability Zones a, b, c, d, e, f
  |
  `-- Edge locations and Regional edge caches (200+ sites)
        used by CloudFront for caching and Route 53 for DNS
```

Choose a Region by latency to your users, which services are available there, the price in that Region, and any data residency rule you must follow.

Availability Zone names are randomised per account: `us-east-1a` in one account may be a different physical zone from `us-east-1a` in another, so never hard-code a zone name across accounts.

The six advantages of cloud computing

Advantage	What it means day to day
Trade fixed expense for variable expense	pay for what you use instead of buying hardware up front
Benefit from massive economies of scale	lower unit cost because AWS runs at aggregate scale
Stop guessing capacity	scale up or down in minutes instead of provisioning for a peak
Increase speed and agility	a fresh server is minutes away, not months
Stop spending on running data centres	focus engineering on what makes the product different
Go global in minutes	deploy into many Regions quickly

The shared responsibility model

Security is a joint duty drawn along the line of who controls each layer.

output

```
  Customer  ...... security IN the cloud .......
            guest OS patches, application, IAM, security
            group rules, data, encryption, configuration
  ---------------------------------------------- ---- service boundary
  AWS      ...... security OF the cloud .......
            facilities, Regions, Availability Zones, hosts,
            network infrastructure, virtualisation layer
```

The line moves with the service model:

Service	AWS manages	Customer manages
EC2 (IaaS)	facility, host, hypervisor, network	guest OS, patches, app, security groups, IAM, data
RDS (managed)	everything under the DB engine, engine patching	database users, schema, parameters, network access, data
S3 and Lambda (abstracted)	the whole platform	data, access policies, encryption, function code

1.3 Examples

Example 1: sort each layer of a stack by owner.

output

```
Layer                     Owned by AWS   Owned by customer
--------------------------------------------------------------------
Physical building         yes            no
Host hardware             yes            no
Virtualisation hypervisor yes            no
Guest OS on EC2           no             yes
Application code          no             yes
IAM users, groups, roles  no             yes
Security group rules      no             yes
RDS engine patching       yes            no
S3 public access block    no             yes
```

Example 2: choose a Region from constraints.

output

```
Constraint                Decision
------------------------------------------------------------
Users in Ireland and EU   eu-west-1 keeps data inside the EU
Lowest price in the US    us-east-1 is usually the cheapest
Newest services first     pick a Region that already lists them
Disaster recovery         pair eu-west-1 with a second Region
```

Example 3: list the Regions offered to an account (illustrative, not executed).

shell

```
aws ec2 describe-regions --query "Regions[].RegionName" --output table
```

output

```
(illustrative output, not executed here)
-----------------
| DescribeRegions |
+-----------------+
|  eu-west-1      |
|  us-east-1      |
+-----------------+
```

Example 4: a workload described as a short inventory you can apply the model to.

JSON

```
{
  "workload": "photo-uploader",
  "components": [
    { "service": "EC2", "region": "eu-west-1", "customerDuty": "patch the Amazon Linux guest OS" },
    { "service": "S3",  "region": "eu-west-1", "customerDuty": "block public access, set bucket policy"
    { "service": "RDS", "region": "eu-west-1", "customerDuty": "manage database users and schema" }
  ]
}
```

output

```
(valid JSON. This is a description you would write, not a value a
command returns. Save it as inventory.json and reuse it in exercise 5.)
```

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating security as an AWS-only job	the guest OS, data and IAM stay wide open	read every workload through the shared responsibility model
Running everything in one Availability Zone	a single zone failure takes the service offline	spread instances and replicas over at least two zones
Hard-coding Availability Zone names	the letters differ between accounts and launches fail	list zones with `describe-availability-zones`, choose dynamically
Picking a Region on cost alone	a cheap Region can be far from users or miss a service	weigh latency, service availability, cost and data residency together
Using the root login for daily work	root can disable protections and is one point of failure	create an admin IAM user, secure root with MFA, then store it away
Ignoring data residency rules	keeping regulated data outside its Region is legal exposure	tag data by sensitivity and pin it to compliant Regions

1.5 Practice exercises

1. List the six advantages of cloud computing that AWS publishes.
2. Name two duties that stay with the customer even on a fully managed service such as Amazon RDS.
3. Distinguish a Region from an Availability Zone in two sentences.
4. Why is it unsafe to share an Availability Zone name such as `eu-west-1a` between two AWS accounts?
5. For an Amazon S3 bucket that stores customer photos, name three settings that belong to the customer side of the shared responsibility model.

Solutions

1. Variable instead of fixed cost; economies of scale; stop guessing capacity; increase speed and agility; stop running data centres; go global in minutes.
2. Any two of: managing the database users and permissions, the schema and queries, the parameter settings, the network access rules, and the confidentiality of the data itself.
3. A Region is a named geographic area with its own services and prices, such as `eu-west-1`. An Availability Zone is one or more independent data centres inside that Region, each with its own power, cooling and network.
4. Zone letters map to physical zones at random per account, so the same letter can point to a different facility in another account. Any resource or latency assumption built on the shared name becomes wrong.
5. Any three of: turning on Block Public Access and leaving it on, writing a bucket policy that grants least privilege, choosing server-side encryption, enabling versioning, and setting a lifecycle rule.

1.6 Mini test

1. What is the default pricing model of cloud computing? A) Pay a fixed monthly fee per server B) Pay-as-you-go for what you consume C) Pay a one-time licence fee D) Pay only for storage
2. Which unit is the largest geographic grouping of AWS resources? A) Availability Zone B) Edge location C) Region D) Subnet
3. True or false: one Availability Zone consists of exactly one data centre and nothing more.
4. In the shared responsibility model, who patches the guest operating system of an Amazon EC2 instance? A) AWS, always B) The customer C) The hardware vendor D) No one
5. Which two duties remain with the customer on a fully managed service such as Amazon RDS? A) Racking the servers and cabling the network B) The data stored and the access granted to it C) Replacing failed disks D) Cooling the data centre
6. Why should you not hard-code an Availability Zone name across two AWS accounts? A) Zone letters are assigned at random per account and may be different facilities B) It is against the terms of service C) Zone names change every day D) Zone names only work in one Region
7. Which AWS service uses edge locations to cache content close to users? A) Amazon EC2 B) Amazon S3 C) Amazon CloudFront D) Amazon RDS
8. At a minimum, how many Availability Zones does a Region contain? A) One B) Two C) Five D) Ten

Answer key and explanations

1. B - cloud computing charges for consumption rather than for capacity owned in advance.
2. C - a Region is the top-level geographic grouping; zones and subnets sit inside it or inside a VPC.
3. False - an Availability Zone can hold one or more data centres, and the count is neither exposed nor fixed.

4. B - with EC2 the customer controls the guest operating system and so patches it; AWS owns everything below.
5. B - even when AWS operates the platform, the customer still owns the data and decides who may reach it.
6. A - the mapping from a zone letter to a physical facility is per account, so shared assumptions break.
7. C - CloudFront is the content delivery network that keeps copies at edge locations.
8. B - every Region has at least two Availability Zones; larger Regions have more.

Chapter 2 - Identity and access management: users, roles and policies

2.1 Summary

- IAM is the service that decides who can do what in an AWS account, and there is no charge for it.
- A principal is an identity that can send a request: a user, a group, a role, or an AWS service acting on your behalf.
- The root user is created with the account; secure it with a strong password and MFA, then set it aside.
- IAM users hold long-term credentials: a password for the console and, optionally, access keys for the command line and APIs.
- Groups collect users so a permission can be granted once to everyone who needs it; a group cannot hold another group.
- A role holds no long-term credentials and hands out temporary credentials through the Security Token Service when it is assumed.
- A policy is a JSON document made of statements that each allow or deny a set of actions on a set of resources.
- Identity-based policies attach to users, groups and roles, while resource-based policies attach to resources such as buckets, queues and key policies.
- A request is allowed only when an applicable policy allows it and no policy denies it; the starting point is always deny.
- Least privilege means granting only the actions and resources a principal truly needs, and revisiting that grant over time.

2.2 Concepts

Principals and identities

Everything a person or a service does starts with a principal that signs a request.

output

```
                     request an action
    human ---------> +---------------+ --------> AWS APIs
    workload ----> |    IAM      |
    AWS service -> | user/group/role |
                    +---------------+
```

Identity	Credential it holds	Typical use
Root user	email plus password	account setup, billing, and closing the account
IAM user	console password, and access keys for the CLI	a specific person or app that needs standing access
Group	none; members inherit the group policies	give many users the same permissions at once
Role	temporary credentials issued by STS	AWS services, cross-account access, and federated sign-in

A user is one identity with its own credentials. A role is an identity that anyone who is allowed to assume can wear for a while, and it never stores a password or a static key.

Users, groups and roles

Create a user for a person or an application that must hold credentials. Put the user in one or more groups so you can change a permission in one place for everyone. Prefer a role over a user whenever the access ends, whenever a service needs to call another service, or whenever someone outside the account must reach into it, because roles expire on their own.

Policy structure

An identity-based or resource-based policy is a JSON object with a `Version` string and a `Statement` array. Each statement is one rule.

JSON

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucketForReports",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::example-reports"
    },
    {
      "Sid": "ReadReportObjects",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example-reports/*"
    }
  ]
}
```

Field	Meaning
`Sid`	an optional label for the statement
`Effect`	`Allow` or `Deny`
`Action`	the API calls the rule applies to, for example `s3:GetObject`
`Resource`	the ARNs the rule applies to, with `*` matching any or all
`Principal`	who the statement applies to; used in resource-based and trust policies
`Condition`	extra requirements, such as a source address or a tag, that must also hold

Two statements above combine: the identity can list one named bucket and read the objects inside it, and nothing else in S3.

Policy types and who they belong to

Policy type	Attached to	Used for
Identity-based	user, group, role	the permissions that identity carries
Resource-based	S3 bucket, SQS queue, KMS key, and a role trust policy	who may reach that resource
Permissions boundary	user or role	an outer ceiling that permissions cannot pass
Service control policy	an account or unit in AWS Organizations	a guardrail that applies across accounts
Session policy	a role session	shrink permissions for one temporary session

AWS managed policies are built and maintained by AWS and cover common jobs; customer managed policies are written and owned by you; inline policies live inside a single identity and are best kept small and rare.

How a request is decided

IAM evaluates every request against all applicable policies and returns one answer. The rules are simple and absolute.

output

```
incoming request
      |
      v
any applicable explicit Deny?  ---- yes ---->  DENIED
      | no
      v
any applicable explicit Allow?  ---- yes ---->  still inside SCP and boundary?
      | no                                      | no -> DENIED
      v                                         | yes
   DENIED                                       v
(default deny)                                ALLOWED
```

- The default answer is deny: silence in the policies means the request fails.
- An explicit Deny in any applicable policy always beats every Allow.
- Order does not matter, neither within a statement nor across policies.
- A service control policy and a permissions boundary act as ceilings that an Allow can never rise above.

Wildcards and condition keys

A `*` in an action stands for any run of characters, so `s3:*` covers every S3 API and `s3:Get*` covers the read actions. In a resource ARN a trailing `*` matches the objects under a prefix. A `Condition` narrows a statement with extra requirements, such as a source address or a required tag.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example-reports/*",
      "Condition": { "IpAddress": { "aws:SourceIp": "203.0.113.0/24" } }
    }
  ]
}
```

2.3 Examples

Example 1: an administrator policy that cannot reach billing. The wide Allow is narrowed by the Deny, and the Deny wins.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "*", "Resource": "*" },
    {
      "Effect": "Deny",
      "Action": ["aws-portal:*", "budget:*"],
      "Resource": "*"
    }
  ]
}
```

output

```
(policy accepted by IAM. A least-privilege design would name each needed
Action instead of "*" and drop this example as a shortcut.)
```

Example 2: a trust policy, the resource-based side of a role, that lets Amazon EC2 assume the role.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

output

```
(trust policy accepted. The permissions the role later uses come from a
separate identity-based policy attached to the same role.)
```

Example 3: confirm which identity is calling (illustrative, not executed here).

shell

```
aws sts get-caller-identity
```

output

```
(illustrative output, not executed in the writing environment)
{
    "UserId": "AIDAEXAMPLEUSERID00000",
    "Account": "123456789012",
    "Arn": "arn:aws:iam::123456789012:role/example-ec2-role"
}
```

Example 4: what a role session looks like when it is assumed. The credentials are short lived and cannot be used after `Expiration`.

JSON

```
{
  "Credentials": {
    "AccessKeyId": "ASIAEXAMPLEKEYID0000",
    "SecretAccessKey": "EXAMPLEsecretAccessKeyValueNotReal000000",
    "SessionToken": "EXAMPLEsessionTokenValueNotReal",
    "Expiration": "2026-10-10T12:00:00Z"
  },
  "AssumedRoleUser": {
    "Arn": "arn:aws:sts::123456789012:assumed-role/example-ec2-role/session-1"
  }
}
```

output

```
(shape of the response; every value here is a placeholder, not a real key.)
```

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using the root user for daily work	root can change almost anything, including disabling controls	secure root with MFA, create an admin user, set root aside

Mistake	Why it hurts	Best practice
Sharing one access key between people	one leaked key is a shared breach with no owner	one user per person, rotate keys, prefer roles and SSO
Using long-term keys when a role fits	keys never expire and leak into source code	hand out roles and let STS issue short-lived credentials
Granting `Action: "*"` widely	a single mistake anywhere grows into a total compromise	write least-privilege policies and review them on a schedule
Forgetting that Deny always wins	a well-meant Deny silently breaks a working tool	look for Deny statements first whenever an access call fails
Leaving MFA off on privileged users	a stolen password is enough to sign in	require MFA, above all for console sign-in by admins

2.5 Practice exercises

1. In one sentence, give a case where an IAM role is a better choice than an IAM user.
2. A request matches one policy statement that allows it and another that denies it. What is the outcome, and why?
3. Name the two top-level parts of an IAM policy, and the two fields that every statement must contain.
4. A policy allows `s3:GetObject` on `arn:aws:s3:::example-reports/*`. May the same identity call `s3:ListBucket` on `example-reports`? Explain.
5. What is the difference between an AWS managed policy and a customer managed policy?

Solutions

1. Use a role whenever the access is temporary or is taken by a service or a trusted external identity, because a role issues short-lived credentials through STS and stores no password or static key.
2. The request is denied. An explicit Deny in any applicable policy overrides every Allow, no matter where each statement appears.
3. Top level: `Version` and `Statement`. Every statement must contain an `Effect` and an `Action`; a `Resource` is also required whenever the action supports resource-level permissions, which most do.
4. No. The Allow names only the `s3:GetObject` action on objects under that bucket, written with the trailing `*`. `s3:ListBucket` is a different action on the bucket itself, so it needs its own Allow.
5. An AWS managed policy is created and maintained by AWS for common jobs; a customer managed policy is written and owned by you and can be attached to many identities in the account.

2.6 Mini test

1. Which credential does an IAM role issue when it is assumed? A) A long-term secret access key B) Temporary credentials valid for a limited time C) A console password D) An API key that never expires
2. What is the result when no policy mentions the action a request asks for? A) Allow, because nothing blocks it B) Deny, because nothing allows it C) Allow if the user is an admin D) Ask the root user
3. What is the job of an IAM group? A) It holds temporary credentials B) It collects users so one policy can reach all of them C) It stores other groups D) It replaces an IAM role
4. True or false: an IAM group can contain another IAM group.
5. Which policy field states whether a request is permitted or refused? A) `Action` B) `Resource` C) `Effect` D) `Sid`
6. Which policy type attaches to an Amazon S3 bucket to name who may read it? A) An identity-based policy B) A resource-based bucket policy C) A permissions boundary D) A session policy
7. A request matches one Allow and one Deny. What is the result? A) Allow, because the Allow is listed first B) Deny, because an explicit Deny always wins C) Whichever policy was evaluated last D) It depends on the order of the statements
8. Which service issues the temporary credentials used to assume a role? A) AWS KMS B) AWS STS C) Amazon Cognito D) Amazon CloudFront

Answer key and explanations

1. B - the Security Token Service returns temporary credentials that carry an expiry set by the session length.
2. B - evaluation starts from deny, and only an explicit Allow opens a door.
3. B - groups exist to share one policy across many users at the same time.
4. False - group membership is flat; groups cannot be nested.
5. C - `Effect` holds the value `Allow` or `Deny`.
6. B - a bucket policy is a resource-based policy attached to the bucket itself.
7. B - an explicit Deny overrides every Allow regardless of order or position.
8. B - AWS Security Token Service, seen as the `sts` calls, issues and refreshes role credentials.

Chapter 3 - Core compute on EC2 with pricing basics

3.1 Summary

- Amazon EC2 provides resizable virtual servers, called instances, under the IaaS model.
- An Amazon Machine Image is a template that carries the operating system, the software and the block-device mapping used to launch an instance.

- Instance types group families by purpose, such as compute, memory, storage or GPU, and each family offers sizes that differ in CPU, memory and networking.
- An instance moves through a lifecycle from pending to running, then to stopped, shutting-down and terminated.
- The host layer gives an instance its CPU, memory, local disks and basic metrics, while everything inside the guest belongs to the customer.
- On-Demand instances are paid for by the second or hour with no commitment, which makes them flexible and the most costly per unit.
- Reserved Instances buy a one or three year term for a large discount, while Savings Plans commit to a spending level instead of a fixed configuration.
- Spot Instances draw on spare capacity for a steep discount and can be reclaimed with short notice, so they suit interruptible work.
- Each instance lives in exactly one Region and one Availability Zone, and it uses a key pair to prove the identity of its first administrator.
- The size, the Region and the purchase option chosen together decide most of the EC2 charge.

3.2 Concepts

What an instance is

An instance is a virtual machine. You choose an image, a size, a network and a storage layout, and AWS runs it on a physical host while keeping it isolated from other tenants. You control the guest operating system and everything installed on it; AWS controls the host, the hypervisor and the underlying hardware.

Instance types and naming

A type is written as a family, a generation, optional letters for the chip or feature, and a size. For example `m7g.2xlarge` is a balanced, Arm-based, general-purpose instance with eight vCPUs and twice the memory of `.large`.

Family	Optimised for	Typical use
T (`t3`, `t4g`)	burstable general purpose	small web servers and dev boxes
M (`m6i`, `m7g`)	balanced general purpose	application and mid-size servers
C (`c7g`, `c7i`)	compute	batch jobs, gaming, scientific modelling
R (`r7g`, `r7i`)	memory	in-memory caches and databases
I (`i4i`)	local storage	NoSQL stores and analytics
G, P, Inf	GPU and machine learning inference	graphics, training and inference

T instances accumulate CPU credits while idle and spend them in bursts, which suits a workload that is quiet most of the day.

Instance storage

Storage	Location	Behaviour
Instance store	disks attached to the physical host	very fast, but lost when the instance stops or ends
EBS volume	a network volume inside one Availability Zone	survives a stop, can be snapshotted and reattached

An EBS-backed root volume can be stopped and started again with its data intact. An instance-store-backed instance cannot be stopped, only rebooted or terminated.

The instance lifecycle

output

```
launch
   |
   v
pending ---> running ---> shutting-down ---> terminated
   ^            |
   |            +--> stopped --(start)--> running
   |                       stopped keeps EBS, pays no compute
reboot:    running ---> running   (stays on the same host)
```

A running instance is billed for every second it is up. A stopped EBS-backed instance pays nothing for compute but still pays for its EBS volumes. A terminated instance cannot be recovered.

Metadata and the key pair

Every instance can read facts about itself from the link-local metadata service at `169.254.169.254`. Use the token-oriented mode, called IMDSv2, which is more resistant to certain web attacks. At launch you choose a key pair: AWS installs the public half, and you keep the private half to sign in over SSH on Linux or to unlock the administrator password on Windows.

The five ways to pay

Option	Commitment	Typical saving	Interruption	Best for
On-Demand	none	baseline price	not interrupted	short spikes and first workloads
Reserved Instance	1 or 3 year	large, up to about 72 percent	steady	a known, steady baseline
Savings Plan	1 or 3 year of spend	large and flexible	steady	usage that may shift family or Region
Spot	none; uses spare capacity	up to about 90 percent	a two minute notice before reclaim	fault-tolerant and stateless work
Dedicated Host	per physical host	varies	a server reserved for you	bring-your-own-licence needs

Match the option to the shape of the workload: a steady baseline earns a discount from a Reserved Instance or a Savings Plan, variable demand stays On-Demand, and interruptible batch work fits Spot. A Reserved Instance is tied to a fixed configuration, while a Savings Plan commits to a spend in dollars per hour and applies more loosely.

New accounts receive a Free Tier allowance of 750 hours per month of a small instance (`t2.micro`, or `t3.micro` where the Region supports it) for the first 12 months.

3.3 Examples

Example 1: read the vCPU and memory of a type (illustrative, not executed here).

shell

```
aws ec2 describe-instance-types --instance-types t3.micro --query "InstanceTypes[0].{VCPUs:VCpuInfo.Def
```

JSON

```
{
    "VCPUs": 2,
    "MemoryMiB": 1024
}
```

Example 2: match a workload to a purchase option.

output

```
Workload shape                          Purchase option
-----------------------------------------------------------------
Steady web tier all day                 Savings Plan or Reserved Instance
A monthly training run you can restart Spot
An experiment that lasts one afternoon On-Demand
Oracle licence bound to physical cores Dedicated Host
```

Example 3: a launch request written as JSON for `--cli-input-json`.

JSON

```
{
  "ImageId": "ami-0EXAMPLE00000000",
  "InstanceType": "t3.micro",
  "MinCount": 1,
  "MaxCount": 1,
  "KeyName": "example-key",
  "SecurityGroupIds": ["sg-0EXAMPLE0000000000"]
}
```

output

```
(passed to run-instances as --cli-input-json; every id is a placeholder and
this request was not executed in the writing environment.)
```

Example 4: fetch metadata with IMDSv2 from inside the instance (illustrative).

shell

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-secon
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-type
```

output

```
(illustrative output, meant to run on the instance itself, not executed here)
t3.micro
```

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Choosing an oversized type by habit	you pay for idle CPU and memory every hour	start small, measure the load, then resize
Leaving test instances running	a forgotten instance bills continuously	stop or terminate what is idle and schedule the rest
Using On-Demand for a steady baseline	the per-unit rate stays the highest	buy a Savings Plan or a Reserved Instance for the base
Discarding the private key at launch	some sign-in paths are lost for good	store the private key safely at creation time
Keeping durable data on instance store	the data vanishes when the instance ends	place durable data on EBS or S3
Running a database on Spot or a long job on Spot	an interruption stalls work or risks data loss	reserve Spot for stateless, fault-tolerant tasks

3.5 Practice exercises

1. Name the three parts that make up an Amazon Machine Image.
2. An EBS-backed instance is stopped for two days. Which charges continue, and which stop?
3. Distinguish a Reserved Instance from a Savings Plan in one sentence.
4. Why is Spot capacity a poor fit for a primary database?
5. Which storage survives the end of an instance, instance store or EBS, and why?

Solutions

1. A template for the root volume, the launch permissions that decide who may use the image, and a block-device mapping that lists the volumes to attach.
2. The compute charge stops while the instance is stopped; the charge for the EBS volumes continues, and an Elastic IP that is kept also continues to bill.
3. A Reserved Instance reserves capacity for a fixed configuration, while a Savings Plan commits to a spend measured in dollars per hour and applies more loosely across the instance family, Region or service.
4. Spot instances can be reclaimed with a two minute notice, so a primary database faces abrupt unavailability and, without the right safeguards, data loss; Spot suits fault-tolerant, stateless work instead.
5. EBS survives, because it is a separate network volume that persists until it is deleted; instance store data is tied to the host and is lost when the instance ends.

3.6 Mini test

1. What is an Amazon Machine Image? A) A running virtual server B) A template with the OS, software and volume mapping used to launch an instance C) A saved log file D) A network firewall rule
2. Which instance family is optimised for memory? A) C B) M C) R D) I
3. When an instance is stopped, which charge continues? A) The per-second CPU charge B) The charge for its EBS volumes C) All charges stop together D) The Spot discount is refunded
4. What happens to instance store data when an instance terminates? A) It becomes a snapshot B) It is lost C) It turns into an EBS volume D) It is archived for a year
5. Which purchase option can be interrupted with a two minute notice? A) On-Demand B) Reserved Instance C) Spot Instance D) Dedicated Host
6. True or false: both a Savings Plan and a Reserved Instance need a one or three year commitment.
7. Where does an instance read facts about itself at the address `169.254.169.254`? A) Amazon CloudWatch B) The instance metadata service C) Amazon Route 53 D) AWS Systems Manager
8. What is the purpose of the key pair chosen at launch? A) To encrypt the EBS volume B) To prove the identity of the first administrator at sign-in C) To set the security group D) To choose the Availability Zone

Answer key and explanations

1. B - the image is the template an instance is created from, not the running instance.
2. C - the R family is memory optimised; C is compute and I is storage.
3. B - a stopped EBS-backed instance halts compute charges yet keeps paying for its volumes.
4. B - instance store is ephemeral and bound to the host machine.
5. C - Spot draws on spare capacity and may be reclaimed with short notice.
6. True - both use a one or three year term; they differ in what the commitment covers.
7. B - the link-local metadata service serves facts about the running instance.
8. B - AWS installs the public key, and the matching private key proves the identity of the person signing in.

Chapter 4 - Storage with S3 and Glacier

4.1 Summary

- Amazon S3 is object storage designed to hold and serve any amount of data from anywhere over the internet.

- Data is kept as objects inside buckets, where an object is the data plus metadata and a key, and a bucket is a container that lives in one Region.
- The namespace inside a bucket is flat, so the slashes in a key only create the look of folders.
- Whole objects are written and read, with no in-place edit, so a new upload replaces the object as a whole.
- Durability is designed to reach 99.999999999 percent, called eleven nines, by spreading copies across devices and facilities.
- Storage classes trade access speed and a minimum kept duration for a lower price, from S3 Standard to Glacier Deep Archive.
- Lifecycle rules shift objects between classes and expire them on a schedule, which trims cost with no manual steps.
- Versioning keeps older copies of an object, which helps recovery after an accidental overwrite or deletion.
- The S3 Glacier storage classes serve archives kept for months or years and seldom read, with retrieval that spans milliseconds to hours.
- Access control starts with blocking all public access and then writing least-privilege IAM and bucket policies.

4.2 Concepts

Objects and buckets

An object is the unit of storage: it is the bytes, a key or name, and a block of metadata. A bucket is a container for objects, created in a Region, with a name that is unique across all of AWS. A single object may be as large as 5 TB.

output

```
Amazon S3
  |
  |-- Bucket: example-reports           Region: eu-west-1
  |     |-- 2026/q1.pdf                 object = data + metadata + key
  |     |-- 2026/q2.pdf
  |     `-- assets/logo.png             the slashes are part of the key
  |
  `-- Bucket: example-backups           a second Region when replication is on
```

Because the space is flat, there is no move or rename; copying to a new key and deleting the old key is a move. Large uploads use multipart upload, which sends the object in parts and reassembles it.

Consistency and durability

S3 gives strong read-after-write consistency: a successful write is immediately visible to a following read, including a listing. Durability comes from redundancy across at least three Availability Zones for most classes, so the loss of a facility does not remove an object. Availability, the share of time the object can be read, is what differs between classes.

Storage classes

Class	Access pattern	Minimum kept	Retrieval
S3 Standard	frequent	none	milliseconds
S3 Intelligent-Tiering	unknown or changing	none	milliseconds, with a small monitoring fee
S3 Standard-IA	infrequent, needs milliseconds	30 days	milliseconds
S3 One Zone-IA	infrequent, re-creatable data	30 days	milliseconds, one zone only
S3 Glacier Instant Retrieval	rare, needs milliseconds	90 days	milliseconds
S3 Glacier Flexible Retrieval	backup and archive	90 days	minutes to hours
S3 Glacier Deep Archive	years of cold archive	180 days	up to about 12 hours in bulk

Classes with a minimum kept duration charge for that duration even if an object is deleted sooner, so a class must match how long the object will stay.

Key features

Feature	What it does
Versioning	keeps every version of an object in a bucket
Lifecycle rules	move objects between classes and expire them on a schedule
Cross-Region Replication	copies new objects to a bucket in another Region
Object Lock	a write-once-read-many lock that blocks deletion for a set period
Server-side encryption	encrypts at rest with SSE-S3, SSE-KMS or SSE-C
Static website hosting	serves objects directly as a web site
Presigned URL	a time-limited link that grants access to a single object

Encryption and access control

Encrypt in transit with TLS, and encrypt at rest with server-side encryption: SSE-S3 uses keys managed by S3, SSE-KMS uses a key you control in AWS KMS, and SSE-C uses a key you supply with each request. On the access side, keep Block Public Access on, then grant permission with least privilege through IAM policies and a bucket policy. Bucket ACLs are a legacy mechanism and are best left disabled.

The S3 Glacier archive classes

Archive storage is reached through the S3 Glacier storage classes listed above. The S3 Glacier service also exposes a separate interface of vaults, archives and retrieval jobs, with Expedited, Standard and Bulk tiers, for teams that manage archives outside of S3. Current

AWS guidance points most new work at the S3 Glacier storage classes inside S3, so that is what this volume relies on.

4.3 Examples

Example 1: a bucket policy that allows one role to read and refuses any insecure connection.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadForApp",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:role/app-role" },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example-reports/*"
    },
    {
      "Sid": "DenyInsecureTransport",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": ["arn:aws:s3:::example-reports", "arn:aws:s3:::example-reports/*"],
      "Condition": { "Bool": { "aws:SecureTransport": "false" } }
    }
  ]
}
```

output

```
(valid bucket policy JSON. The Deny statement wins over the Allow whenever a
request arrives without TLS, which is why order between them does not matter.)
```

Example 2: a lifecycle rule that cools and then retires objects.

JSON

```
{
  "Rules": [
    {
      "ID": "archive-old-reports",
      "Status": "Enabled",
      "Filter": { "Prefix": "2025/" },
      "Transitions": [
        { "Days": 30, "StorageClass": "STANDARD_IA" },
        { "Days": 90, "StorageClass": "GLACIER" }
      ],
      "Expiration": { "Days": 365 }
    }
  ]
}
```

output

```
(a lifecycle configuration: objects under 2025/ move to STANDARD_IA after 30
days, to GLACIER after 90 days, and are deleted after 365 days.)
```

Example 3: upload a file, then list the prefix (illustrative, not executed here).

shell

```
aws s3 cp report.pdf s3://example-reports/2026/q2.pdf
aws s3 ls s3://example-reports/2026/
```

output

```
(illustrative output, not executed in the writing environment)
2026-03-31 09:12:00      90352 2026/q1.pdf
2026-06-30 12:04:11      84211 2026/q2.pdf
```

Example 4: the body of a restore request for an object stored in a Glacier class.

JSON

```
{
  "Days": 7,
  "GlacierJobParameters": { "Tier": "Standard" }
}
```

output

```
(a restore request passed to the restore action; the object is readable for
the copied days and then returns to an archive class. Not executed here.)
```

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Exposing a bucket for public read	sensitive objects leak across the internet	keep Block Public Access on at account and bucket level
Choosing Deep Archive for hot data	retrieval is slow and adds a request fee	match the storage class to how often the object is read
Deleting an object and expecting recovery	without versioning the object cannot return	enable versioning on buckets that matter
Ignoring the minimum kept duration	an early deletion still bills the whole minimum	only transition data that stays put for that long
Picking a bucket name already in use	names are global and the creation fails	include the account or project in the name
Uploading the same large file many times	requests and parts add cost	use multipart upload and lifecycle rules to manage size

4.5 Practice exercises

1. Name the two parts of an object besides the data itself.
2. A file will not be opened for five years but must be kept for legal reasons. Which storage class fits, and what is the trade-off?
3. What does versioning protect against, and what does it not protect against on its own?

4. A bucket must refuse any request that is not sent over TLS. Which policy element handles that?
5. Why is there no true folder structure inside an S3 bucket?

Solutions

1. A key, the name of the object, and a block of metadata. When versioning is enabled there is also a version identifier attached to each copy.
2. S3 Glacier Deep Archive, the cheapest class for years of cold storage. The trade-off is slow retrieval, since bulk retrieval can take up to about 12 hours, together with a 180 day minimum kept duration.
3. Versioning protects against accidental overwrite and deletion by keeping prior versions. On its own it does not stop a caller who deletes the versions themselves, and each copy adds storage cost, so it is often paired with Object Lock when the data must never change.
4. A Deny statement with a Condition on `aws:SecureTransport` set to `false` covering `s3:*`, which refuses every action that arrives without TLS.
5. The namespace inside a bucket is flat; folders are only an illusion built from slashes in a key, and there is no folder object to rename or move.

4.6 Mini test

1. What is the maximum size of a single Amazon S3 object? A) 500 GB B) 5 TB C) 160 GB D) There is no limit
2. Which class gives milliseconds access to very rarely read data with a 90 day minimum? A) S3 Standard-IA B) S3 Glacier Instant Retrieval C) S3 Glacier Flexible Retrieval D) S3 Glacier Deep Archive
3. What does a lifecycle rule do? A) Deletes a bucket B) Moves objects between classes and expires them on a schedule C) Encrypts objects at upload D) Copies objects to another account
4. Why is there no true rename of an S3 object? A) Buckets are read-only B) The namespace is flat and the whole object is stored by its key, so a rename is a copy then a delete C) Only the console can rename D) Keys are fixed at bucket creation
5. Which server-side option uses a key you control in AWS KMS? A) SSE-S3 B) SSE-KMS C) SSE-C D) TLS
6. True or false: with versioning enabled, every deletion can be undone afterwards.
7. What is the purpose of a presigned URL? A) To publish a web site B) To grant time-limited access to a single object without AWS credentials C) To replicate a bucket D) To lock an object
8. Which Glacier Deep Archive retrieval tier may take about 12 hours? A) Expedited B) Standard C) Bulk D) On-Demand

Answer key and explanations

1. B - one object may be as large as 5 TB; larger data uses multipart upload or splits into many objects.

2. B - Glacier Instant Retrieval returns in milliseconds yet keeps a 90 day minimum; Flexible and Deep Archive are slower and cheaper.
3. B - lifecycle rules automate the transitions between classes and the final expiration.
4. B - S3 has a flat namespace; an object is written and read by its whole key, so a rename is a copy followed by a delete.
5. B - SSE-KMS encrypts at rest with a key that lives in AWS KMS and that you control.
6. False - versioning keeps earlier versions, but a caller who deletes those versions is not stopped, so pair it with controls such as Object Lock when the data must never change.
7. B - a presigned URL carries a signature that authorizes one action on one object until the link expires.
8. C - Bulk retrieval on Deep Archive can take up to about 12 hours; Expedited is not offered for Deep Archive.

Chapter 5 - Networking with VPC, subnets and security groups

5.1 Summary

- A VPC is a virtual network that you define inside one AWS Region to run resources in an isolated slice of the cloud.
- You define a VPC by its IPv4 address space, written as a CIDR block such as 10.0.0.0/16.
- A subnet is a smaller range of the VPC that belongs to one Availability Zone and holds the resources you place in it.
- A route table decides where traffic leaving a subnet is sent, and each subnet is tied to exactly one route table.
- An internet gateway connects a VPC to the internet and so enables public subnets, while a NAT gateway lets private subnets reach outbound without allowing connections in.
- A security group is a stateful firewall that controls traffic to and from an instance, and it supports only allow rules.
- A network ACL is a stateless firewall that controls traffic to and from a subnet, and it holds allow and deny rules judged in number order.
- Public subnets send outbound traffic to an internet gateway, while private subnets send it through a NAT gateway and keep their instances off the public internet.
- VPC endpoints keep traffic between a VPC and services such as S3 and DynamoDB on the AWS network, with no internet gateway in the path.
- Each Region ships with a default VPC ready for public subnets, but production designs normally define their own from scratch.

5.2 Concepts

CIDR blocks and subnets

A VPC is created with one primary IPv4 CIDR block. A subnet carves a piece of that range and pins it to a single Availability Zone. A `/16` block holds 65,536 addresses and a `/24` holds 256, so planning the split early saves rework later.

output

```
VPC  10.0.0.0/16   (Region eu-west-1)
  |
  |-- Availability Zone eu-west-1a
  |     |-- public-subnet   10.0.1.0/24   routes outbound to the internet gateway
  |     `-- private-subnet  10.0.3.0/24   routes outbound to the NAT gateway
  |
  `-- Availability Zone eu-west-1b
        |-- public-subnet   10.0.2.0/24   routes outbound to the internet gateway
        `-- private-subnet  10.0.4.0/24   routes outbound to the NAT gateway
```

AWS reserves the first four and the last address of every subnet for its own use, so a `/24` offers 251 addresses for resources.

Whether a subnet is public or private is not a property of the subnet itself; it is decided by the route table it uses. A subnet whose route table has a route of `0.0.0.0/0` to an internet gateway is public. A subnet whose default route points to a NAT gateway is private.

Routers, gateways and addresses

Component	Purpose
Route table	maps a destination CIDR to a target, with an automatic local route inside the VPC
Internet gateway	attaches to the VPC and provides a path to the internet, scaled by AWS
NAT gateway	a managed device in a public subnet that lets private instances reach the internet, but only outbound
Elastic IP	a static public IPv4 address that you keep in the account until it is released

Place one NAT gateway in one public subnet per Availability Zone that holds private subnets, so a single gateway is not shared across zones.

Security groups

A security group acts as a stateful firewall at the network interface of a resource. It starts by denying all inbound traffic and allowing all outbound. You add allow rules, and because the group is stateful, a permitted inbound request is answered automatically without a matching outbound rule. You can point a rule at another security group by its id, which lets only members of that group connect. There are no deny rules, and all the rules of all attached groups are combined.

Network ACLs

A network ACL is a stateless guard at the edge of a subnet. It holds numbered rules that are read from lowest to highest, and each rule either allows or denies a matched packet. Because it is stateless, a permitted inbound request needs a matching rule that allows the reply traffic back out. Every subnet uses one ACL, and a custom ACL starts closed.

Security groups versus network ACLs

Aspect	Security group	Network ACL
Applies to	an instance or elastic network interface	a single subnet
State	stateful, replies flow automatically	stateless, replies need their own rule
Rules	allow only	allow and deny
Order	all rules are weighed together	read by rule number, lowest first
Best use	the everyday firewall on a resource	an extra guard at the subnet boundary

A common design uses both: a tight security group on each resource and a broader network ACL to catch anything unexpected at the subnet edge.

VPC endpoints and flow logs

A gateway endpoint for S3 or DynamoDB is added as a route target in a route table and carries no extra hourly charge, so traffic to those services never leaves the AWS network. An interface endpoint, powered by AWS PrivateLink, creates an elastic network interface with a private IP address for a supported service and is charged per hour and by data moved. Flow logs capture accepted and rejected IP traffic into CloudWatch Logs or Amazon S3 for troubleshooting and as a security record.

The default VPC

Every Region ships with a default VPC that holds a `/16` block, one public subnet per Availability Zone, an internet gateway, a main route table, and a default security group and network ACL. It is handy for experiments, while production designs usually define a named VPC with documented subnets and route tables.

5.3 Examples

Example 1: the CIDR plan for a small design.

output

```
Resource           CIDR             Zone            Kind
-------------------------------------------------------------------
vpc-main           10.0.0.0/16      eu-west-1       spans every zone
subnet-public-a    10.0.1.0/24      eu-west-1a      public
subnet-public-b    10.0.2.0/24      eu-west-1b      public
subnet-private-a   10.0.3.0/24      eu-west-1a      private
subnet-private-b   10.0.4.0/24      eu-west-1b      private
```

Example 2: make a VPC and one public subnet (illustrative, not executed here).

shell

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-subnet --vpc-id vpc-0EXAMPLE00000000 --cidr-block 10.0.1.0/24 --availability-zone eu-wes
```

output

```
(illustrative only; every id is a placeholder and nothing ran while writing.)
```

Example 3: add an inbound rule to a security group (illustrative, not run here).

shell

```
aws ec2 authorize-security-group-ingress --group-id sg-0EXAMPLE0000000000 --protocol tcp --port 443 --c
```

output

```
(illustrative: this opens inbound HTTPS to any address. In production, name a
narrower source such as the load balancer security group.)
```

Example 4: the route table behind a public subnet, sketched as JSON.

JSON

```
{
  "RouteTable": [
    { "DestinationCidrBlock": "10.0.0.0/16", "Target": "local" },
    { "DestinationCidrBlock": "0.0.0.0/0", "Target": "igw-0EXAMPLE000000" }
  ]
}
```

output

```
(a sketch of the routes in a public subnet: traffic inside the VPC stays local
and everything else goes to the internet gateway. Not executed here.)
```

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
One subnet for the whole design	a single failure or a flat blast radius takes all down	give every zone a public and a private subnet
Opening the security group to the world on port 22	the whole internet can reach the port	allow SSH from a bastion or AWS Systems Manager only
Sharing one NAT gateway across every zone	a single failure cuts all outbound access	place one NAT gateway per zone with private subnets
Forgetting the reply rule on a network ACL	a stateless guard drops the answer and sessions stall	add the matching outbound rule for the ephemeral ports
Putting a database in a public subnet	the instance gains a public reachable address	keep data stores in private subnets with no public address
Using the default VPC for production	names and rules are vague and easy to misuse	build a named VPC that follows a documented plan

5.5 Practice exercises

1. What decides whether a subnet is public or private?
2. A security group holds only allow rules. How, then, do you shut out one specific client?
3. Why must a network ACL list a rule for return traffic when a security group does not?

4. Name the two kinds of VPC endpoint, and say which keeps traffic off the internet with no separate hourly charge.
5. How many usable addresses does a `/24` subnet offer for resources, and why is that number not 256?

Solutions

1. The route table it is tied to. A default route of `0.0.0.0/0` to an internet gateway makes a subnet public, while a default route to a NAT gateway makes it private.
2. You never deny in a security group; you simply do not add an allow for that client, and anything without a matching allow is already blocked. When a specific block is required, use a network ACL with a deny rule.
3. A network ACL is stateless, so it judges each packet by itself and cannot remember a prior permitted request; a rule must allow the reply back out. A security group is stateful and tracks the connection, letting replies flow automatically.
4. Gateway endpoints and interface endpoints. A gateway endpoint for S3 or DynamoDB keeps traffic off the internet with no separate hourly charge, while an interface endpoint uses an elastic network interface and is billed per hour and by data.
5. It offers 251. AWS reserves the first four addresses and the last one of each subnet for its own purposes, which leaves 251 of the 256.

5.6 Mini test

1. What makes a subnet public rather than private? A) The name it is given B) A route to an internet gateway in its route table C) The Availability Zone it sits in D) The number of instances inside it
2. Which statement about security groups is true? A) They are stateless and support deny rules B) They are stateful and support only allow rules C) They evaluate their rules by number D) They attach to a whole subnet
3. What does a NAT gateway provide? A) Inbound access to a private instance from the internet B) Outbound-only internet access for private subnets C) A static private address D) A path between two VPCs
4. Which VPC endpoint type carries no separate hourly charge? A) Interface endpoint B) Gateway endpoint for S3 or DynamoDB C) A PrivateLink endpoint for EC2 D) A transit gateway attachment
5. True or false: a network ACL can hold a deny rule.
6. Why does a stateless guard need a rule for the reply traffic?
7. How many usable IPv4 addresses does AWS leave in a `/24` subnet? A) 256 B) 251 C) 250 D) 252
8. What does an Elastic IP provide? A) A static public IPv4 address held until you release it B) A second private subnet C) A public DNS name that cannot change D) An extra elastic network interface

Answer key and explanations

1. B - it is the `0.0.0.0/0` route to an internet gateway that turns a subnet into a public one.
2. B - security groups are stateful and carry only allow rules; blocking is work left to a network ACL.
3. B - a NAT gateway lets private instances start outbound connections while it refuses unsolicited inbound ones.
4. B - gateway endpoints for S3 and DynamoDB cost nothing to attach; interface endpoints are billed by the hour and by data.
5. True - a network ACL holds both allow and deny rules, unlike a security group.
6. Because it weighs each packet on its own and cannot remember an earlier permitted request, so a separate rule must let the reply return.
7. B - the reserved first four and last address leave 251 of the 256 for resources.
8. A - an Elastic IP is a public IPv4 address that stays in the account until it is released, which suits a fixed public endpoint.

Chapter 6 - Databases with RDS and DynamoDB

6.1 Summary

- Amazon RDS is a managed service for relational databases, running engines such as MySQL, PostgreSQL, MariaDB, Oracle, SQL Server and Amazon Aurora.
- With RDS, AWS handles provisioning, patching, backup and failover, while the customer keeps the schema, the queries, the database users and the access rules.
- Multi-AZ keeps a synchronous standby in a second Availability Zone and fails over automatically, which protects availability without adding read capacity.
- Read replicas are asynchronous copies that scale read traffic and can be promoted into a primary when needed.
- Automated backups allow a point-in-time recovery inside the retention window, while manual snapshots stay until they are deleted.
- Amazon Aurora is a cloud-native engine, compatible with MySQL and PostgreSQL, that spreads storage across three zones and supports many read replicas.
- Amazon ElastiCache offers an in-memory cache with Redis or Memcached to lift common reads off the database and to hold sessions.
- Amazon DynamoDB is a fully managed NoSQL database that answers in single-digit milliseconds and runs no server for you.
- A DynamoDB table holds items and attributes, with a primary key that is either a single partition key or a partition key plus a sort key.
- DynamoDB capacity is either on-demand or provisioned, with secondary indexes to serve more query patterns and Streams to react to changes.

6.2 Concepts

What RDS manages for you

A managed relational service removes the undifferentiated work of running a database, but not the work of designing and using one.

AWS manages	Customer manages
hardware, operating system and the engine install	database users, roles and grants
engine patching, on a schedule you choose	schema design and the queries that run
automated backup into Amazon S3	parameters that shape how the app behaves
Multi-AZ failover and the replication plumbing	the security groups and other network access
storage growth and replace of failed hardware	the confidentiality of the data itself

High availability and read scaling

output

```
Multi-AZ                          Read replicas
------------------------------    ------------------------------
primary (zone A)                  primary (zone A)
   |  synchronous                    |  asynchronous
   v                                 v
standby (zone B)                  replica (zone B), replica (zone C)
fails over automatically           reads only; can be promoted
```

Multi-AZ is a resilience feature: the standby is not open for reads and its only job is to take over when the primary fails. Read scaling is a separate feature: a read replica serves queries and can become a primary, and it sits in the same or a different Region.

Backups and snapshots

Automated backups run on a window you set and support point-in-time recovery inside a retention window of 0 to 35 days. A manual snapshot is taken on demand and is kept until you delete it, and it can be copied to another Region. Every restore, from either kind, creates a brand new instance with a new address rather than rolling the old one back in place.

Aurora and ElastiCache

Amazon Aurora keeps a storage layer that spans three Availability Zones and grows in steps as it fills, and it allows up to 15 read replicas; Aurora Serverless adjusts capacity by itself for light or variable loads. Amazon ElastiCache runs either Redis, which adds replication and persistence, or Memcached, which is a plain, scalable memory cache; both offload read traffic and can hold user sessions.

DynamoDB basics

DynamoDB is a NoSQL key-value and document store. A table holds items, and an item holds attributes, with no fixed shape, so one item may carry fields that another does not. An item cannot exceed 400 KB.

output

```
Table "products"   (each item is its own set of attributes)
  item A  productId A1  name kettle   price 24.99  inStock true
  item B  productId A2  name toaster  price 39.99  colour red

Primary key choices
  simple     productId alone is the partition key
  composite  productId (partition key) plus model (sort key)
```

The partition key spreads the data across partitions for even load. A sort key keeps a range in order so that range queries can walk one partition. Because there are no joins, a table is shaped by the questions it must answer, and related data is often written together in one item.

DynamoDB offers two capacity modes. On-demand bills per request and suits unknown traffic; provisioned asks you to set read and write capacity units and can scale them automatically. Secondary indexes broaden the queries without a second table: a global secondary index may use a new partition and sort key, while a local secondary index keeps the same partition key and adds a different sort key. DynamoDB Streams record changes in order, which pairs with Lambda to react to updates, and global tables spread active-active copies across Regions.

6.3 Examples

Example 1: one item described as plain JSON.

JSON

```
{
  "productId": "A1",
  "name": "kettle",
  "price": 24.99,
  "inStock": true,
  "tags": ["kitchen", "small-appliance"]
}
```

output

```
(the shape of one item; these are example values, not a live response.)
```

Example 2: the same item in the typed notation DynamoDB commands accept.

JSON

```
{
  "productId": { "S": "A1" },
  "name": { "S": "kettle" },
  "price": { "N": "24.99" },
  "inStock": { "BOOL": true }
}
```

output

```
(S marks a string, N a number, and BOOL a boolean. Save this as item.json for
the next example. Nothing here was executed in the writing environment.)
```

Example 3: write and then read that item (illustrative, not executed here).

shell

```shell
aws dynamodb put-item --table-name products --item file://item.json
aws dynamodb get-item --table-name products --key file://key.json
```

output

```
(illustrative output for the get, not executed in the writing environment)
{
    "Item": {
        "productId": { "S": "A1" },
        "name": { "S": "kettle" },
        "price": { "N": "24.99" },
        "inStock": { "BOOL": true }
    }
}
```

Example 4: choose the right store for the job.

output

```
What you need                    Choose
-----------------------------------------------------------
Joins and transactions           Amazon RDS (relational)
Key lookups at very large scale  Amazon DynamoDB
A shared memory cache            Amazon ElastiCache
Large blobs and files            Amazon S3
```

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating Multi-AZ as a read-scaling tool	the standby serves no reads and only cost rises	add read replicas when you need to scale reads
Keeping every snapshot forever	storage grows quietly month by month	set a retention plan and delete snapshots no longer needed
Sharing one security group across app and database	anything in the group can reach the port	split the roles and let only the app reach the database port
Running with backups turned off	an accidental delete leaves no path back	enable automated backups with point-in-time recovery
Modeling DynamoDB as a relational table	the access patterns DynamoDB is built for are lost	design the table from the queries it must answer
Over-provisioning DynamoDB write units	RCU and WCU are billed whether used or not	start on-demand, watch the load, then set provisioned with auto scaling

6.5 Practice exercises

1. Name three tasks Amazon RDS shoulders that a self-managed database would leave to you.
2. State the difference between Multi-AZ and a read replica.

3. State the difference between an automated backup and a manual snapshot.
4. In DynamoDB, what does the partition key do, and what does adding a sort key enable?
5. A team expects a DynamoDB table whose traffic is unpredictable. Which capacity mode should they start with, and why?

Solutions

1. Any three of: provisioning and maintaining the hardware and the operating system, installing and patching the database engine, running automated backups, and handling Multi-AZ failover with the replication plumbing.
2. Multi-AZ keeps a synchronous standby for automatic failover that protects availability and serves no reads. A read replica is an asynchronous copy that serves read traffic to scale it and can be promoted to a primary.
3. An automated backup is taken on a schedule inside a retention window and supports point-in-time recovery; it is managed by RDS. A manual snapshot is created on demand, is kept until the customer deletes it, and can be copied to another Region.
4. The partition key spreads items across partitions for even load and locates a single item. Adding a sort key orders items that share the partition key, which enables range queries such as greater-than or between inside that partition.
5. On-demand, because it charges per request and needs no capacity planning, which suits traffic that is unknown while the team learns the real pattern. Once the load is stable they can move to provisioned with auto scaling.

6.6 Mini test

1. Which of these is NOT a managed Amazon RDS engine? A) MySQL B) PostgreSQL C) MongoDB D) MariaDB
2. What does Multi-AZ provide? A) Extra read capacity B) A synchronous standby for automatic failover C) Cross-Region replication only D) A cache in front of the database
3. What does a read replica do by default? A) Accepts both writes and reads B) Serves read traffic asynchronously C) Blocks all traffic until promoted D) Replaces the primary every night
4. Which recovery method restores a database to any second inside a window? A) A manual snapshot B) Automated backups with point-in-time recovery C) A read replica D) Object Lock
5. In DynamoDB, what is a sort key used for? A) To encrypt an item B) To order items that share a partition key and allow range queries C) To name a table D) To choose a Region
6. True or false: a DynamoDB primary key can be a partition key only, or a partition key plus a sort key.
7. Which in-memory cache products does Amazon ElastiCache offer? A) Memcached only B) Redis or Memcached C) Varnish D) Hazelcast
8. Which store fits a workload that needs joins and complex transactions? A) DynamoDB B) Amazon RDS C) Amazon S3 D) Amazon ElastiCache

Answer key and explanations

1. C - MongoDB is not an RDS engine; RDS runs relational engines plus Amazon Aurora.
2. B - Multi-AZ exists for availability through a synchronous standby, not for read scaling.
3. B - a read replica is an asynchronous copy that serves reads and that can be promoted later.
4. B - automated backups with point-in-time recovery restore to any point inside the retention window.
5. B - the sort key orders items within a partition and enables range queries on them.
6. True - a simple primary key is one partition key, while a composite key adds a sort key.
7. B - ElastiCache runs either Redis or Memcached.
8. B - relational work with joins belongs on a managed relational engine such as Amazon RDS.